```
‘use strict';

console.log('CAMPUS IT SUPPORT REQUEST MANAGER');

class Stack {
  constructor() {
    this.items = [];
  }

  push(element) {
    this.items.push(element);
  }

  pop() {
    return this.items.pop();
  }

  peek() {
    return this.items[this.items.length - 1];
  }

  isEmpty() {
    return this.items.length === 0;
  }

  size() {
    return this.items.length;
  }

  toString() {
    return this.items.join(', ');
  }
}

console.log('\nPART A: UNDO HISTORY');

const actionHistory = new Stack();

actionHistory.push('Opened Ticket #101');
actionHistory.push('Assigned Ticket #101');
actionHistory.push('Closed Ticket #101');

console.log('Top Action:', actionHistory.peek());
console.log('Undo Action:', actionHistory.pop());
console.log('Remaining Actions:', actionHistory.toString());
console.log('Stack Size:', actionHistory.size());

class Queue {
```

```
  constructor() {
    this.items = [];
  }

  enqueue(element) {
    this.items.push(element);
  }

  dequeue() {
    return this.items.shift();
  }

  front() {
    return this.items[0];
  }

  isEmpty() {
    return this.items.length === 0;
  }

  size() {
    return this.items.length;
  }

  toString() {
    return this.items.join(', ');
  }
}

console.log('\nPART B: WAITING REQUESTS');

const waitingRequests = new Queue();

waitingRequests.enqueue('Ticket #201 - Password Reset');
waitingRequests.enqueue('Ticket #202 - No Internet');
waitingRequests.enqueue('Ticket #203 - Printer Problem');
waitingRequests.enqueue('Ticket #204 - Software Installation');

console.log('Next Request:', waitingRequests.front());
console.log('Processed:', waitingRequests.dequeue());
console.log('Processed:', waitingRequests.dequeue());
console.log('Remaining Requests:', waitingRequests.toString());
console.log('Queue Size:', waitingRequests.size());

class PriorityQueue {
  constructor() {
    this.items = [];
  }
```

```
  enqueue(request, priority) {
    const item = {
      request: request,
      priority: priority
    };

    let inserted = false;

    for (let i = 0; i < this.items.length; i++) {
      if (priority < this.items[i].priority) {
        this.items.splice(i, 0, item);
        inserted = true;
        break;
      }
    }

    if (!inserted) {
      this.items.push(item);
    }
  }

  dequeue() {
    return this.items.shift();
  }

  size() {
    return this.items.length;
  }

  print() {
    this.items.forEach(item => {
      console.log(`${item.request}, Priority ${item.priority}`);
    });
  }
}

console.log('\nPART C: URGENT REQUESTS');

const urgentRequests = new PriorityQueue();

urgentRequests.enqueue('Ticket #301 - Install Browser', 3);
urgentRequests.enqueue('Ticket #302 - Server Offline', 1);
urgentRequests.enqueue('Ticket #303 - Faculty PC Failure', 2);
urgentRequests.enqueue('Ticket #304 - Internet Laboratory Down', 1);

console.log('Initial Priority Order:');
urgentRequests.print();
```

```
const processedPriorityRequest = urgentRequests.dequeue();

console.log(
  'Processed Priority Request:',
  `${processedPriorityRequest.request}, Priority ${processedPriorityRequest.priority}`
);

console.log('Remaining Priority Requests:');
urgentRequests.print();

class Node {
  constructor(element) {
    this.element = element;
    this.next = null;
  }
}

class LinkedList {
  constructor() {
    this.length = 0;
    this.head = null;
  }

  append(element) {
    const node = new Node(element);

    if (this.head === null) {
      this.head = node;
    } else {
      let current = this.head;

      while (current.next !== null) {
        current = current.next;
      }

      current.next = node;
    }

    this.length++;
  }

  insert(position, element) {
    if (position < 0 || position > this.length) {
      return false;
    }

    const node = new Node(element);
```

```
    if (position === 0) {
      node.next = this.head;
      this.head = node;
    } else {
      let previous = this.head;

      for (let i = 1; i < position; i++) {
        previous = previous.next;
      }

      node.next = previous.next;
      previous.next = node;
    }

    this.length++;
    return true;
  }

  removeAt(position) {
    if (position < 0 || position >= this.length) {
      return null;
    }

    let current = this.head;

    if (position === 0) {
      this.head = current.next;
    } else {
      let previous = this.head;

      for (let i = 1; i < position; i++) {
        previous = previous.next;
      }

      current = previous.next;
      previous.next = current.next;
    }

    this.length--;
    return current.element;
  }

  indexOf(element) {
    let current = this.head;
    let index = 0;

    while (current !== null) {
```

```
      if (current.element === element) {
        return index;
      }

      current = current.next;
      index++;
    }

    return -1;
  }

  remove(element) {
    const index = this.indexOf(element);

    if (index === -1) {
      return null;
    }

    return this.removeAt(index);
  }

  isEmpty() {
    return this.length === 0;
  }

  size() {
    return this.length;
  }

  toString() {
    const elements = [];
    let current = this.head;

    while (current !== null) {
      elements.push(current.element);
      current = current.next;
    }

    return elements.join(', ');
  }
}

console.log('\nPART D: COMPLETED REQUESTS');

const completedRequests = new LinkedList();

completedRequests.append('Ticket #401 - Account Activated');
completedRequests.append('Ticket #402 - Printer Connected');
```

```
completedRequests.append('Ticket #403 - Antivirus Updated');

completedRequests.insert(0, 'Ticket #400 - Computer Diagnosed');
completedRequests.insert(
  completedRequests.size(),
  'Ticket #405 - Browser Updated'
);

console.log('Complete List:', completedRequests.toString());
console.log(
  'Index of Ticket #402:',
  completedRequests.indexOf('Ticket #402 - Printer Connected')
);

console.log(
  'Removed at Position 1:',
  completedRequests.removeAt(1)
);

console.log(
  'Removed by Value:',
  completedRequests.remove('Ticket #403 - Antivirus Updated')
);

console.log('Final List:', completedRequests.toString());
console.log('Final Size:', completedRequests.size());
```